

C++ Complete Course

From Fundamentals to Functions — A Developer's Handbook



Saksham Sontakke
github.com/Sakkkky

33+
Source Files

700+
Lines of Code

12
Core Topics

Table of Contents

01	Introduction to C++ History, architecture, what C++ is
02	Variables & Data Types Scope, primitives, built-in types
03	Operators Comparison, logical, arithmetic operators
04	Control Flow if/else, switch, loops
05	Pointers & References Addresses, dereference, pointer-to-pointer
06	Arrays Declaration, access, memory layout
07	Structures & Unions Custom data types, typedef, memory
08	Enums Enumerated types as custom data
09	Functions Deep Dive Prototyping, inline, static, default args
10	Recursion Factorial, Fibonacci — self-calling functions
11	Function Overloading Compile-time dispatch by signature
12	C++ Cheat Sheet Quick reference: types, keywords, containers

CHAPTER 01

Introduction to C++

The language that runs the world — from silicon to software.

What is C++?

C++ is a **general-purpose, compiled, statically typed** programming language. It provides zero-cost abstractions, direct hardware access, multiple paradigms (procedural, OOP, generic, functional), and deterministic performance — no garbage collector, no runtime surprises.

Why C++?

C++ occupies a unique position: it is both a **high-level** language capable of expressing abstract algorithms, and a **low-level** language that can write bare-metal firmware. It powers game engines, browsers, databases, AI backends, trading systems, compilers, and operating systems.

History & Standards Timeline

Standard	Nickname	Key Features
C++98	Foundation	Templates, STL, exceptions, namespaces
C++11	Revolution	auto, lambdas, nullptr, move semantics, smart pointers
C++14	Polish	Generic lambdas, auto return, std::make_unique
C++17	Practical	std::optional, structured bindings, if constexpr
C++20	Paradigm Shift	Concepts, coroutines, modules, ranges, std::format
C++23	Quality of Life	std::expected, std::print, std::flat_map

Your First C++ Program

```
C++
#include <iostream>
using namespace std;

int main() {
    cout << "hello world"; // print to screen
    return 0; // successful exit
}
```

`#include <iostream>` — adds input/output functionality (header file).

`using namespace std` — access all standard library functions without `std::` prefix.

`int main()` — the entry point of every C++ program.

`cout <<` — the insertion operator sends data to the output stream.

return 0 — signals successful execution to the OS.

CHAPTER 02

Variables & Data Types

Scope, primitives, global vs local, built-in sizes.

Declaring Variables

```
C++  
  
int intVar1 = 5, intVar2 = 10;  
float floatVar = 3.14;  
double doubleVar = 3.12345678; // more precision  
char charVar = 'a';  
bool boolVar = true;
```

Primitive Data Types

Type	Size	Range / Notes
short	2 bytes	-32,768 to 32,767
int	4 bytes	-2,147,483,648 to 2,147,483,647
long long	8 bytes	$\pm 9.22 \times 10^{18}$
float	4 bytes	~7 decimal digits of precision
double	8 bytes	~15 decimal digits (default for literals)
long double	12 bytes	Higher precision than double
char	1 byte	Single ASCII character, e.g. 'A'
bool	1 byte	true (1) or false (0)

Variable Scope

Variables declared **outside all functions** are *global* — accessible everywhere. Variables declared **inside a function** are *local* — they exist only within that block.

```
C++  
  
int globalVar = 10; // global  
  
void testFunc() {  
    globalVar = 20; // modifies global  
}  
  
int main() {  
    int localVar = 30; // local – only here  
    cout << ::globalVar; // '::' accesses global directly  
}
```

```
}
```

Float & Double Literals

By default any decimal value (e.g. 34.5) uses **double** (8 bytes). Append **f** for float (4 bytes). Append **L** for long double (12 bytes).

CHAPTER 03

Operators

Arithmetic, comparison, logical, increment, and type-casting.

Arithmetic Operators

```
C++

int a = 10, b = 3;
cout << a + b; // 13 addition
cout << a - b; // 7 subtraction
cout << a * b; // 30 multiplication
cout << a / b; // 3 integer division
cout << a % b; // 1 modulus (remainder)
cout << a++; // 10 post-increment (print then +1)
cout << ++a; // 12 pre-increment (+1 then print)
```

Comparison Operators

Comparison expressions return **1 (true)** or **0 (false)**.

Operator	Meaning	Example (a=10, b=30)
==	Equal to	a==b → 0
!=	Not equal to	a!=b → 1
>	Greater than	a>b → 0
<	Less than	a<b → 1
>=	Greater than or equal	a>=b → 0
<=	Less than or equal	a<=b → 1

Logical Operators

Operator	Symbol	Description
AND	&&	True only when ALL conditions are true
OR		True when AT LEAST ONE condition is true
NOT	!	Reverses the condition (true→false, false→true)

Type Casting

```
C++
```

```
int num1 = 45;
float num2 = 45.56;

cout << (float)num1; // 45.0 - C-style cast
cout << float(num1); // 45.0 - functional cast (same)
cout << (int)num2; // 45 - truncates decimal

int c = int(num2); // store as int
cout << num1 + int(num2); // 90
```


CHAPTER 04

Control Flow

if/else, switch/case, for, while, do-while, break, continue.

if / else if / else

```
C++  
  
int age;  
cin >> age;  
  
if (age > 18) {  
    cout << "You can drive";  
} else if (age == 18) {  
    cout << "Get a license first!";  
} else {  
    cout << "Not eligible";  
}
```

switch / case

```
C++  
  
switch (age) {  
    case 18:  
        cout << "Exactly 18!";  
        break; // EXIT switch block  
    case 20:  
        cout << "You are 20";  
        break;  
    default:  
        cout << "No match found";  
        break;  
}
```

■ Always Use break

Without **break**, execution falls through to subsequent cases. This is intentional in rare cases but almost always a bug.

Loops

```
C++  
  
// for loop – known iteration count
```

```
for (int i = 1; i <= 50; i++) { cout << i; }

// while loop – condition checked BEFORE each iteration
int i = 1;
while (i <= 50) { cout << i; i++; }

// do-while – runs AT LEAST once
int j = 1;
do { cout << j; j++; } while (j <= 50);
```

break & continue

```
C++

// break – exits the loop entirely
for (int i = 0; i <= 40; i++) {
    if (i == 2) break; // stops at 0, 1, 2
    cout << i;
}

// continue – skips current iteration
for (int k = 0; k <= 40; k++) {
    if (k == 2) continue; // skips printing 2
    cout << k;
}
```

CHAPTER 05

Pointers & References

Memory addresses, dereference, pointer-to-pointer, reference variables.

What is a Pointer?

A pointer is a variable that **stores the memory address** of another variable. Every variable occupies a location in RAM; a pointer holds that location.

```
C++
int a = 3;
int* b = &a; // 'b' holds the address of 'a'

cout << &a; // prints address, e.g. 0x7ffd4abc
cout << b; // same address as &a
cout << *b; // 3 – dereference: value at address

// Pointer-to-pointer
int** c = &b;
cout << *c; // address stored in b
cout << **c; // 3 – value at address of address
```

Reference Variables

A reference is an **alias** for an existing variable. Both names refer to the same memory location.

```
C++
float x = 500;
float& y = x; // y is a reference to x

cout << x; // 500
cout << y; // 500 – same memory!

y = 999;
cout << x; // 999 – x changed too
```

CHAPTER 06

Arrays

Collections of same-type elements in contiguous memory.

Declaring & Accessing Arrays

```
C++  
  
// syntax: datatype name[size] = {values};  
int arr[4] = {11, 22, 33, 44};  
  
cout << arr[0]; // 11 (index from 0)  
cout << arr[3]; // 44  
  
// Iterating with for loop  
for (int i = 0; i < 4; i++) {  
    cout << arr[i] << endl;  
}
```

■ ■ Out-of-Bounds Access

Accessing beyond array size (e.g. `arr[4]` on a 4-element array) does NOT throw an error in C++. Instead it returns a **garbage value from nearby memory** — one of the most dangerous bugs in low-level programming.

Pointer Arithmetic on Arrays

The array name itself stores the address of its **first element**. You can traverse elements using pointer arithmetic.

```
C++  
  
int marks[4] = {55, 66, 77, 88};  
int* p = marks; // p points to marks[0]  
  
cout << *p; // 55 — marks[0]  
cout << *(p + 1); // 66 — marks[1]  
cout << *(p + 2); // 77 — marks[2]  
cout << *(p + 3); // 88 — marks[3]  
  
// Pointer arithmetic formula:  
// address(p+i) = address(p) + i * sizeof(dataType)
```

CHAPTER 07

Structures & Unions

Custom data types combining multiple fields of different types.

Structures (struct)

A **struct** groups variables of different types under one name. Memory is allocated for *all* fields.

```
C++  
  
struct Employee {  
    int eId;  
    char name;  
    float salary;  
};  
  
int main() {  
    struct Employee sakky;  
    sakky.eId = 12;  
    sakky.name = 's';  
    sakky.salary = 5000000;  
    cout << sakky.name; // s  
}
```

typedef Shorthand

```
C++  
  
typedef struct NewEmployee {  
    int eId;  
    char name;  
    float salary;  
} Emp;  
  
Emp e1; // instead of 'struct NewEmployee e1'  
e1.eId = 19;
```

Unions

A **union** allocates memory for only *one* field at a time — the size of the union is the size of its largest member. Using more than one field simultaneously produces a **garbage value**.

Feature	struct	union
Memory	Allocated for ALL fields	Allocated for ONE (largest) field
Usage	Access all fields freely	Only one field at a time

Size	Sum of all field sizes	Size of largest field
------	------------------------	-----------------------

CHAPTER 08

Enums

Named integer constants — turning numbers into readable identifiers.

Declaring & Using Enums

C++

```
enum Meals { breakfast, lunch, dinner };  
// breakfast = 0, lunch = 1, dinner = 2 (implicit)  
  
cout << breakfast; // 0  
cout << lunch; // 1  
cout << dinner; // 2  
  
// 'Meals' becomes a usable data type:  
Meals m1 = breakfast;  
Meals m2 = dinner;  
cout << m2; // 2
```

Enums as Data Types

After declaring `enum Meals { ... }`, you can use **Meals** as a data type just like `int` or `char`. This makes code more readable and less error-prone than using raw integers.

CHAPTER 09

Functions Deep Dive

Prototyping, call by value vs reference, inline, static, default arguments.

Basic Functions

```
C++

// Returns a value — must use 'return'
int sum(int a, int b) {
    return a + b;
}

// void — no return needed
void greet() {
    cout << "Hello!";
}
```

Function Prototyping

C++ reads top-to-bottom. If a function is defined *below* where it's called, you must declare its **prototype** above `main()`.

```
C++

int sub(int a, int b); // prototype (declaration)

int main() {
    cout << sub(10, 3); // works — compiler knows signature
}

int sub(int a, int b) { return a - b; } // definition below
```

Call by Value vs. Call by Reference

Method	What's Passed	Can Modify Original?
Call by Value	A copy of the variable	No — original unchanged
Call by Reference (pointer)	Address of variable	Yes — modifies original
Call by Reference (C++ var)	Alias of variable	Yes — modifies original

```
C++

// CALL BY REFERENCE using pointer
void swapPtr(int* a, int* b) {
```



```
int temp = *a; *a = *b; *b = temp;
}

swapPtr(&x, &y); // passes addresses

// CALL BY REFERENCE using C++ reference var
void swapRef(int& a, int& b) {
    int temp = a; a = b; b = temp;
}

swapRef(x, y); // cleaner syntax, same effect
```

Inline & Static Functions

C++

```
// inline – reduces call overhead by copying body at call site
// Tradeoff: faster but uses more memory
inline int product(int a, int b) { return a * b; }

// static local variable – initialized ONCE, persists across calls
int counter() {
    static int c = 0; // only runs first call
    c = c + 1;
    return c;
}

// counter() -> 1, counter() -> 2, counter() -> 3 ...
```

Default Arguments

C++

```
// Default args must be at the END (rightmost)
float calcInterest(int salary, float rate = 1.04) {
    return salary * rate;
}

calcInterest(12000); // uses rate = 1.04
calcInterest(12000, 1.0); // overrides default
```

CHAPTER 10

Recursion

Functions that call themselves — factorial, Fibonacci, and the call stack.

What is Recursion?

Recursion is when a function calls **itself** directly or indirectly. Every recursive function needs a **base case** to stop the recursion, otherwise it becomes an infinite loop and causes a stack overflow.

Factorial

```
C++  
  
int factorial(int n) {  
    if (n <= 1) return 1; // base case  
    return n * factorial(n - 1); // recursive call  
}  
  
// factorial(6) = 6 * 5 * 4 * 3 * 2 * 1 = 720
```

Fibonacci Series

```
C++  
  
// 1, 1, 2, 3, 5, 8, 13, 21, ...  
// f(n) = f(n-2) + f(n-1)  
int fib(int n) {  
    if (n < 2) return 1;  
    return fib(n - 2) + fib(n - 1);  
}  
  
// Note: naive recursion for fib causes overlapping  
// subproblems — use memoization for large n.
```

CHAPTER 11

Function Overloading

Same name, different signatures — compile-time dispatch.

How It Works

C++ allows **multiple functions with the same name** as long as they differ in number or types of arguments. The compiler automatically picks the right one.

```
C++  
  
int sum(int a, int b) {  
    cout << "2 args";  
    return a + b;  
}  
  
int sum(int a, int b, int c) {  
    cout << "3 args";  
    return a + b + c;  
}  
  
sum(4, 5); // calls 2-arg version  
sum(4, 5, 6); // calls 3-arg version
```

Volume Calculator — Overloaded Functions

```
C++  
  
int volumeCylinder(int r, int h) { return 3.14*r*r*h; }  
int volumeCuboid(int l, int b, int h) { return l*b*h; }  
int volumeCube(int a) { return a*a*a; }  
  
cout << volumeCylinder(2, 4); // 50  
cout << volumeCuboid(2, 4, 10); // 80  
cout << volumeCube(2); // 8
```

CHAPTER 12

C++ Quick Reference

Data types, keywords, operators, STL containers, and algorithm complexity.

I/O Operators

Symbol	Name	Usage
<<	Insertion operator	cout << value — output
>>	Extraction operator	cin >> variable — input
cout	Standard output	cout << "Hello";
cin	Standard input	cin >> x;
endl	End of line	Newline + flush buffer
getline()	Line input	getline(cin, str); — reads full line

Reserved Keywords (Key Subset)

Group	Keywords
Data Types	bool char int short long float double void signed unsigned
Control	if else switch case default break continue goto return
Loops	for while do
OOP	class struct private protected public virtual this friend
Memory	new delete
Storage	auto register static extern mutable const constexpr
Exceptions	try catch throw

STL Container Complexity

Container	Access	Insert Back	Search
vector	O(1)	O(1) amort	O(n)
deque	O(1)	O(1)	O(n)
list	O(n)	O(1)	O(n)
map	O(log n)	O(log n)	O(log n)
unordered_map	O(1) avg	O(1) avg	O(1) avg
set	—	O(log n)	O(log n)

priority_queue	$O(1)$ top	$O(\log n)$	—
----------------	------------	-------------	---

Common Mistakes

■ Don't	■ Do Instead
Use &arrayName to get address	Array name already IS the address
Access arr[n] on n-element array	Stay within bounds (0 to n-1)
Forget break in switch	Always use break to prevent fall-through
Define function below main without prototype	Add prototype above main()
Use multiple union fields	Only use one union field at a time

End of Course Material

This document covers all 33 source files from the C++ course repository.
Topics span from basic I/O to function overloading — the core of procedural C++.

Built with care by **Saksham Sontakke**
github.com/Sakkkky • [cpp-course](#)